

Quarto Manuals

Tinashe M. Tapera¹, Chris¹, Danielle^{2,3}, and Michelle⁴

¹Department of Psychology, Ana and Blanca's University

²Danielle's Primary Affiliation

³Danielle's Secondary Affiliation

⁴Buffalo, NY

Author Note

Tinashe M. Tapera  <https://orcid.org/0000-0001-9080-5010>

Chris  <https://orcid.org/0000-0000-0000-0002>

Danielle  <https://orcid.org/0000-0000-0000-0003>

Michelle  <https://orcid.org/0000-0000-0000-0004>

Carina Mengano is now at Generic University.

The authors have no conflicts of interest to disclose. Author roles were classified using the Contributor Role Taxonomy (CRediT; credit.niso.org) as follows: Tinashe M. Tapera: conceptualization, writing; Chris: project administration, formal analysis; Danielle: formal analysis, writing; Michelle: writing, methodology, formal analysis

Correspondence concerning this article should be addressed to Tinashe M. Tapera, Department of Psychology, Ana and Blanca's University, 1234 Capital St., Albany, NY 12084-1234, USA, Email: ttapera@hsph.harvard.edu

Abstract

to be fleshed out from bullet form below

Impact Statement

Scientific programming workflows are becoming increasingly complex, while existing support mechanisms remain poorly matched to that complexity. Traditional documentation is flexible but non-executable, command-line wrappers are executable but often too rigid, and current agentic systems are promising but not yet sufficiently reliable for many high-stakes workflow setup tasks. We propose Quarto Manuals, an executable documentation framework built on Quarto that helps authors create flexible, scalable, and reproducible workflow manuals composed of structured pages with explicit procedures and checks. Powered by the quarto-emit backend, Quarto Manuals enable authors to create executable documentation that allows users to conveniently step through scientific computing workflows while generating reproducible project artifacts and receiving immediate feedback on whether each step has been implemented correctly.

Keywords: APA Style, Quarto, Markdown, apaquarto

Quarto Manuals

Abstract

Software development is continuing to expand in both scale and complexity, evidenced by the growth of global repository counts and large-scale software archives([Martinelli, 2026](#); ?) At the same time, AI and agentic systems are reshaping how software is produced, making it increasingly important to preserve accessibility, transparency, and reproducibility in scientific software and data science workflows.

Despite the central role of these workflows in academia and industry, effective, reproducible documentation remains an overlooked and under-served part of delivering high quality software ([Aghajani et al., 2019](#)). This issue is likely to become more acute as the volume and sophistication of AI generated code submitted to public repositories continues to increase ([Robbes et al., 2026](#)). Developers therefore need solutions beyond traditional documentation to support workflow execution — address the limitations of traditional software documentation to executing workflows in a reproducible and verifiable manner as the landscape of software development continues to evolve.

Conventional documentation can describe a workflow, but for complex, multi-step, order-dependent tasks and projects, it is often insufficient capturing and verifying critical runtime execution details such as platform dependencies, configuration, versions, and relationships between steps ([Ebert et al., 2015](#)). Command-line interface (CLI) wrappers can improve automation and reproducibility of a declarative workflow by providing a directly executable interface for complex tasks ([van der Aalst et al., 2009](#)). However, developing and maintaining CLI wrappers can be time-consuming and introduces a persistent trade-off between control and flexibility: highly specialized wrappers can improve consistency, but may be inflexible and difficult to maintain, while more generic wrappers enable complexity by shifting the burden to the user through large numbers of flags and configuration options ([Brack et al., 2022](#); [Sadiq et al., 2005](#)). Lastly, AI and agentic systems are promising, but their reproducibility remains limited by non-determinism ([Siddiq et al., 2025](#)). Furthermore, current practical agentic workflow

implementations (such as harnesses) still require careful human oversight to achieve acceptable reliability ([Agrawal et al., 2026](#)).

Executable literate programming approaches can encode and enforce complex workflow steps *and* generate idempotent results. Quarto is a modern, plain-text, multi-language, and multi-output literate programming framework for creating reproducible documents and workflows ([Allaire et al., 2026](#)). Building on Quarto with a carefully designed set of templates and extensions, we introduce **Quarto Manuals**, a framework for executable software manuals that guide users through a workflow as ordered, interactive pages that combine explanation, code execution, and verification. Rather than hiding workflow decisions and steps, Quarto Manuals keep them visible and editable, making them ideal for research computing contexts where local constraints and expert judgment often shape execution parameters. The framework is powered by `quarto-emit`, a lightweight backend extension that materializes conventional workflow artifacts from manual pages when needed. We demonstrate the approach with three examples of increasing sophistication: a simple manual, an intermediate manual, and a complex manual.

For authors, Quarto Manuals provide a structured yet adaptable way to turn recurring workflows into reusable manuals; for operators, they provide a stepwise, testable process for generating reproducible setup artifacts and confirming progress throughout execution in a structured and auditable manner.

Feature Roadmap

- ☒ abstract as prose
- ☐ stylize each block so it is visually distinct
- ☐ use Lua to print out useful render messages when blocks are being processed
- ☐ negative reinforcement: identify and correct misuse of the manual visually
- ☐ setup a testing suite in a separate repo
- ☐ can Lua be used to connect checks and prereqs across pages? Maybe?

References

References marked with an asterisk indicate studies included in the meta-analysis.

- Aghajani, E., Nagy, C., Vega-Márquez, O. L., Linares-Vásquez, M., Moreno, L., Bavota, G., & Lanza, M. (2019). Software Documentation Issues Unveiled. *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 1199–1210.
<https://doi.org/10.1109/ICSE.2019.00122>
- Agrawal, S., Anadkat, H. B., Athimoolam, K. K., Bhardwaj, H., Chowdhury, T., Gao, S., Kamat, P. K., Makwana, V., Shariff, M. H., Badkul, A., Xie, L., & Sinitskiy, A. V. (2026, January 6). *Can AI Conduct Autonomous Scientific Research? Case Studies on Two Real-World Tasks*. <https://doi.org/10.64898/2026.01.05.697809> (Pre-published)
- Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., Dervieux, C., & Woodhull, G. (2026). *Quarto* (Version 1.10) [Computer software]. <https://doi.org/10.5281/zenodo.5960048>
- Brack, P., Crowther, P., Soiland-Reyes, S., Owen, S., Lowe, D., Williams, A. R., Groom, Q., Dillen, M., Coppens, F., Grüning, B., Eguinoa, I., Ewels, P., & Goble, C. (2022). Ten simple rules for making a software tool workflow-ready. *PLOS Computational Biology*, 18(3), e1009823. <https://doi.org/10.1371/journal.pcbi.1009823>
- Ebert, P., Müller, F., Nordström, K., Lengauer, T., & Schulz, M. H. (2015). A general concept for consistent documentation of computational analyses. *Database: The Journal of Biological Databases and Curation*, 2015, bav050. <https://doi.org/10.1093/database/bav050>
- Martinelli, N. (2026, January 16). *Software Heritage Activity Report: 2025*. Software Heritage. <https://www.softwareheritage.org/2026/01/16/software-heritage-activity-report-2025/>
- Robbes, R., Matricon, T., Degueule, T., Hora, A., & Zacchiroli, S. (2026). Agentic Much? Adoption of Coding Agents on GitHub. *ACM Transactions on Software Engineering and Methodology*. <https://doi.org/10.1145/3822180>
- Sadiq, S. W., Orlowska, M. E., & Sadiq, W. (2005). Specification and validation of process constraints for flexible workflows. *Information Systems*, 30(5), 349–378.
<https://doi.org/10.1016/j.is.2004.05.002>
- Siddiq, M. L., Islam-Gomes, A., Sekerak, N., & Santos, J. C. S. (2025, November 29). *Large Language Models for Software Engineering: A Reproducibility Crisis*.

<https://doi.org/10.48550/arXiv.2512.00651>

van der Aalst, W. M. P., Pesic, M., & Schonenberg, H. (2009). Declarative workflows: Balancing between flexibility and support. *Computer Science - Research and Development*, 23(2), 99–113. <https://doi.org/10.1007/s00450-009-0057-9>